

Real Estate API Integration Guide

Base Configuration

Base URL: `https://bricklyrealestate-production.up.railway.app`

WebSocket URL: `wss://bricklyrealestate-production.up.railway.app`

Authentication: Bearer Token (JWT)

All authenticated endpoints require the `Authorization` header:

```
Authorization: Bearer {access_token}
```

Table of Contents

- 1. [Authentication Endpoints](#)
- 2. [User Management Endpoints](#)
- 3. [Property Endpoints](#)
- 4. [Wishlist Endpoints](#)
- 5. [Booking Endpoints](#)
- 6. [Chat Endpoints](#)
- 7. [WebSocket Real-time Messaging](#)
- 8. [Error Handling](#)
- 9. [Frontend Integration Examples](#)

1. Authentication Endpoints

1.1 Register User

POST `/auth/register`

Creates a new user account. Available roles: `BUYER`, `SELLER`, `ADMIN`

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123",
  "name": "John Doe",
  "role": "BUYER",
  "phone": "+1234567890"
}
```

Response (201):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "John Doe",
    "role": "BUYER",
    "phone": "+1234567890"
  }
}
```

Frontend Action: Store `access_token` in `localStorage/sessionStorage` and `user.id` for future requests.

1.2 Login

POST `/auth/login`

Authenticates existing user and returns access token.

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "John Doe",
    "role": "BUYER"
  }
}
```

Frontend Action: Store `access_token` and user data. Redirect to dashboard.

1.3 Refresh Token

GET `/auth/refresh`

Headers: Authorization: Bearer `{access_token}`

Refreshes the authentication token.

Response (200):

```
{
  "access_token": "new_token_here"
}
```

2. User Management Endpoints

2.1 Get Current User Profile

GET `/users/profile`

Headers: Authorization: Bearer `{access_token}`

Retrieves the authenticated user's profile.

Response (200):

```
{
  "id": "uuid",
  "email": "user@example.com",
  "name": "John Doe",
  "role": "BUYER",
  "phone": "+1234567890",
  "createdAt": "2024-01-01T00:00:00.000Z"
}
```

2.2 Update Profile

PUT `/users/profile`

Headers: Authorization: Bearer `{access_token}`

Request Body:

```
{
  "name": "John Updated",
  "phone": "+1234567899"
}
```

Response (200): Updated user object

2.3 Get All Users (Admin Only)

GET /users

Headers: Authorization: Bearer {access_token}

Role Required: ADMIN

Returns list of all users in the system.

2.4 Delete User (Admin Only)

DELETE /users/{userId}

Headers: Authorization: Bearer {access_token}

Role Required: ADMIN

3. Property Endpoints

3.1 Create Property (Seller Only)

POST /properties

Headers: Authorization: Bearer {access_token}

Role Required: SELLER

Content-Type: multipart/form-data ⚠ (NOT application/json — because you're uploading images)

⚠ **AI Price Prediction:** When you create a property, the backend automatically calls the AI API to predict the optimal price. The response includes both your price and the AI's suggested price.

⚠ **Images:** You must upload **at least 4 images** (max 10). Images are uploaded to Cloudinary and stored as URLs.

Request — Form Data Fields:

Field	Type	Required	Description
images	File[]	⚠ Min 4	Property images (jpg/png/webp)
title	string	⚠	Property title
description	string	⚠	Property description
type	string	⚠	Property type enum
price	number	⚠	Price in EGP
bedrooms	number	⚠	Number of bedrooms
bathrooms	number	⚠	Number of bathrooms
area	number	⚠	Area in m ²
furnished	boolean	⚠	true / false
level	number	⚠	Floor level

Field	Type	Required	Description
paymentOption	string	☑	Payment method
deliveryDate	string	☑	ISO date string
city	string	☑	City name
neighborhood	string	☑	Neighborhood
address	string	☑	Full address
latitude	number	☑	GPS latitude
longitude	number	☑	GPS longitude

Property Types:

- APARTMENT, VILLA, TOWNHOUSE, SINGLE_FAMILY, CONDO, DUPLEX

Response (201):

```
{
  "id": "property-uuid",
  "title": "Luxury Villa in Dubai Marina",
  "status": "PENDING",
  "price": 2500000,
  "images": [
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img1.jpg",
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img2.jpg",
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img3.jpg",
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img4.jpg"
  ],
  "aiPriceSuggested": 2450000,
  "aiPriceDifference": 50000,
  "aiPriceDifferencePercentage": 2.04,
  "type": "VILLA",
  "bedrooms": 4,
  "bathrooms": 3,
  "area": 3500,
  "furnished": true,
  "city": "Dubai",
  "sellerId": "seller-uuid",
  "createdAt": "2024-01-01T00:00:00.000Z"
}
```

Error (400) — Less than 4 images:

```
{
  "statusCode": 400,
  "message": "At least 4 images are required",
  "error": "Bad Request"
}
```

Frontend — How to Send (JavaScript):

```

const formData = new FormData();

// Append images (must be at least 4)
files.forEach(file => formData.append('images', file));

// Append property fields
formData.append('title', 'Luxury Villa in Dubai Marina');
formData.append('description', 'Beautiful 4-bedroom villa with sea view');
formData.append('type', 'VILLA');
formData.append('price', '2500000');
formData.append('bedrooms', '4');
formData.append('bathrooms', '3');
formData.append('area', '3500');
formData.append('furnished', 'true');
formData.append('level', '2');
formData.append('compound', 'Marina Residences');
formData.append('paymentOption', 'Cash/Mortgage');
formData.append('deliveryDate', '2024-12-31');
formData.append('city', 'Dubai');
formData.append('neighborhood', 'Dubai Marina');
formData.append('address', '123 Marina Walk');
formData.append('latitude', '25.0806');
formData.append('longitude', '55.1394');

const token = localStorage.getItem('access_token');

const response = await fetch(
  'https://bricklyrealestate-production.up.railway.app/properties',
  {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`
      // ⚠ Do NOT set Content-Type manually – browser sets it with boundary
    },
    body: formData
  }
);

const property = await response.json();

// Show AI price suggestion
if (property.aiPriceSuggested) {
  const diff = property.aiPriceDifferencePercentage;
  if (diff > 20) showWarning('Your price is above market average');
  else if (diff < -20) showWarning('Your price is below market average');
  else showSuccess('Your price is within market range');
}

```

Frontend — React with Axios:

```
import axios from 'axios';

async function createProperty(formFields, imageFiles) {
  const formData = new FormData();

  // Append images
  imageFiles.forEach(file => formData.append('images', file));

  // Append all other fields
  Object.entries(formFields).forEach(([key, value]) => {
    formData.append(key, value);
  });

  const response = await axios.post('/properties', formData, {
    headers: {
      // ⚠ Let axios set Content-Type automatically for FormData
      'Content-Type': 'multipart/form-data'
    }
  });

  return response.data;
}
```

AI Price Fields Explained:

- `aiPriceSuggested` : AI's recommended price (can be `null` if AI service is unavailable)
- `aiPriceDifference` : Your price – AI price (positive = overpriced, negative = underpriced)
- `aiPriceDifferencePercentage` : % difference from AI suggestion

3.2 Get All Properties (Public)

GET `/properties?page=1&limit=10`

No Authentication Required

Query Parameters:

- `page` (optional): Page number (default: 1)
- `limit` (optional): Items per page (default: 10)
- `type` (optional): Property type filter (APARTMENT, VILLA, TOWNHOUSE, SINGLE_FAMILY, CONDO, DUPLEX)
- `minPrice` (optional): Minimum price
- `maxPrice` (optional): Maximum price
- `bedrooms` (optional): Number of bedrooms
- `bathrooms` (optional): Number of bathrooms
- `city` (optional): City filter
- `neighborhood` (optional): Neighborhood filter
- `status` (optional): Property status (PENDING, APPROVED, REJECTED)
- `furnished` (optional): true/false

Example with Filters:

GET `/properties?type=VILLA&minPrice=1000000&maxPrice=3000000&bedrooms=4&city=Dubai&status=APPROVED`

Response (200):

```
{
  "data": [
    {
      "id": "uuid",
      "title": "Luxury Villa",
      "price": 2500000,
      "aiPriceSuggested": 2450000,
      "images": [
        "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img1.jpg",
        "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img2.jpg",
        "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img3.jpg",
        "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img4.jpg"
      ],
      "type": "VILLA",
      "bedrooms": 4,
      "bathrooms": 3,
      "area": 3500,
      "city": "Dubai",
      "status": "APPROVED",
      "views": 125,
      "seller": {
        "id": "seller-uuid",
        "name": "John Seller",
        "email": "seller@example.com",
        "phone": "+1234567890"
      },
      "createdAt": "2024-01-01T00:00:00.000Z",
      ...
    }
  ],
  "total": 50,
  "page": 1,
  "limit": 10,
  "totalPages": 5
}
```

Frontend Action: Use for property listing pages with pagination and filters.

Note: The `seller` object now includes only `id`, `email`, `name`, and `phone` (not full user data).

3.3 Get Property by ID (Public)

GET `/properties/{propertyId}`

No Authentication Required

Returns detailed information about a specific property. **Automatically increments view count.**

Response (200):

```
{
  "id": "property-uuid",
  "title": "Luxury Villa in Dubai Marina",
  "description": "Beautiful 4-bedroom villa with sea view",
  "price": 2500000,
  "aiPriceSuggested": 2450000,
  "aiPriceDifference": 50000,
  "aiPriceDifferencePercentage": 2.04,
  "images": [
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img1.jpg",
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img2.jpg",
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img3.jpg",
    "https://res.cloudinary.com/dx0n30auq/image/upload/v1/brickly/properties/img4.jpg"
  ],
  "type": "VILLA",
  "bedrooms": 4,
  "bathrooms": 3,
  "area": 3500,
  "furnished": true,
  "level": 2,
  "compound": "Marina Residences",
  "paymentOption": "Cash/Mortgage",
  "deliveryDate": "2024-12-31T00:00:00.000Z",
  "city": "Dubai",
  "neighborhood": "Dubai Marina",
  "address": "123 Marina Walk",
  "latitude": 25.0806,
  "longitude": 55.1394,
  "status": "APPROVED",
  "views": 126,
  "seller": {
    "id": "seller-uuid",
    "name": "John Seller",
    "email": "seller@example.com",
    "phone": "+1234567890"
  },
  "createdAt": "2024-01-01T00:00:00.000Z",
  "updatedAt": "2024-01-02T00:00:00.000Z"
}
```

Frontend Actions:

- 1. Display property details
- 2. Show AI price comparison if available
- 3. Display seller contact info
- 4. Show view count
- 5. Enable contact/booking buttons

3.4 Update Property (Seller Only)

PUT /properties/{propertyId}

Headers: Authorization: Bearer {access_token}

Role Required: SELLER (must be property owner) or ADMIN

Content-Type: multipart/form-data

All fields are optional (partial update). If you include new images , they will replace the existing images.

Request — Form Data Fields (all optional):

Field	Type	Description
images	File[]	New images — replaces all existing images
title	string	Updated title

Field	Type	Description
description	string	Updated description
bedrooms	number	Updated bedrooms
bathrooms	number	Updated bathrooms
area	number	Updated area
furnished	boolean	Updated furnished status
...	...	Any other property field

Frontend Example:

```
const formData = new FormData();

// Only append fields you want to update
formData.append('title', 'Updated Villa Title');
formData.append('price', '2700000');

// Optional: add new images to replace existing ones
newImageFiles.forEach(file => formData.append('images', file));

const response = await fetch(
  `https://bricklyrealestate-production.up.railway.app/properties/${propertyId}`,
  {
    method: 'PUT',
    headers: { 'Authorization': `Bearer ${token}` },
    body: formData
  }
);
```

3.5 Update Property Status (Admin Only)

PUT /properties/{propertyId}/status

Headers: Authorization: Bearer {access_token}

Role Required: ADMIN

Request Body:

```
{
  "status": "APPROVED"
}
```

Status Values: PENDING, APPROVED, REJECTED

3.6 Delete Property

DELETE /properties/{propertyId}

Headers: Authorization: Bearer {access_token}

Role Required: SELLER (owner) or ADMIN

4. Wishlist Endpoints

4.1 Get My Wishlist

GET /wishlists

Headers: Authorization: Bearer {access_token}

Role Required: BUYER

Response (200):

```
[
  {
    "id": "wishlist-uuid",
    "property": {
      "id": "property-uuid",
      "title": "Luxury Villa",
      "price": 2500000,
      ...
    },
    "createdAt": "2024-01-01T00:00:00.000Z"
  }
]
```

4.2 Add to Wishlist

POST /wishlists/{propertyId}

Headers: Authorization: Bearer {access_token}

Role Required: BUYER

Adds a property to the user's wishlist.

Response (201): Wishlist item object

4.3 Remove from Wishlist

DELETE /wishlists/{propertyId}

Headers: Authorization: Bearer {access_token}

Role Required: BUYER

5. Booking Endpoints

5.1 Create Booking

POST /bookings

Headers: Authorization: Bearer {access_token}

Role Required: BUYER

Request Body:

```
{
  "date": "2024-12-15",
  "time": "14:00",
  "notes": "Looking forward to viewing this property",
  "propertyId": "property-uuid"
}
```

Response (201):

```
{
  "id": "booking-uuid",
  "date": "2024-12-15",
  "time": "14:00",
  "status": "PENDING",
  "notes": "Looking forward to viewing this property",
  "property": { ... },
  "buyer": { ... }
}
```

Booking Status: PENDING, CONFIRMED, CANCELLED, COMPLETED

Frontend Action: Store booking.id for status updates.

5.2 Get My Bookings

GET /bookings

Headers: Authorization: Bearer {access_token}

Returns all bookings for the authenticated user (buyer sees their bookings, seller sees bookings for their properties).

5.3 Get Booking by ID

GET /bookings/{bookingId}

Headers: Authorization: Bearer {access_token}

5.4 Update Booking Status

PUT /bookings/{bookingId}/status

Headers: Authorization: Bearer {access_token}

Role Required: SELLER (property owner) or BUYER (booking owner)

Request Body:

```
{
  "status": "CONFIRMED"
}
```

Status Values: PENDING, CONFIRMED, CANCELLED, COMPLETED

6. Chat Endpoints

6.1 Create Chat

POST /chats

Headers: Authorization: Bearer {access_token}

Request Body:

```
{
  "sellerId": "seller-user-id",
  "propertyId": "property-uuid"
}
```

Response (201):

```
{
  "id": "chat-uuid",
  "buyer": { ... },
  "seller": { ... },
  "property": { ... },
  "createdAt": "2024-01-01T00:00:00.000Z"
}
```

Frontend Action: Store `chat.id` for messaging. Check if chat already exists before creating.

6.2 Get My Chats

GET `/chats`

Headers: Authorization: Bearer `{access_token}`

Returns all chats for the authenticated user.

6.3 Get Chat by ID

GET `/chats/{chatId}`

Headers: Authorization: Bearer `{access_token}`

6.4 Get Chat Messages

GET `/chats/{chatId}/messages`

Headers: Authorization: Bearer `{access_token}`

Returns all messages in a specific chat.

Response (200):

```
[
  {
    "id": "message-uuid",
    "content": "Hello, I'm interested in this property",
    "sender": {
      "id": "user-uuid",
      "name": "John Doe"
    },
    "createdAt": "2024-01-01T00:00:00.000Z"
  }
]
```

7. WebSocket Real-time Messaging

7.1 WebSocket Connection

WebSocket URL: `wss://bricklyrealstate-production.up.railway.app`

For Local Development: `ws://localhost:3000`

The WebSocket connection enables real-time chat messaging between buyers and sellers.

7.2 Connection Setup

JavaScript/TypeScript Example:

```

import { io } from 'socket.io-client';

// Connect to WebSocket server
const socket = io('https://bricklyrealestate-production.up.railway.app', {
  auth: {
    token: 'your-jwt-token-here'
  },
  transports: ['websocket', 'polling']
});

// Connection events
socket.on('connect', () => {
  console.log('Connected to WebSocket server');
  console.log('Socket ID:', socket.id);
});

socket.on('disconnect', () => {
  console.log('Disconnected from WebSocket server');
});

socket.on('connect_error', (error) => {
  console.error('Connection error:', error);
});

```

React Hook Example:

```

import { useEffect, useState } from 'react';
import { io } from 'socket.io-client';

function useSocket(token) {
  const [socket, setSocket] = useState(null);
  const [connected, setConnected] = useState(false);

  useEffect(() => {
    if (!token) return;

    const newSocket = io('https://bricklyrealestate-production.up.railway.app', {
      auth: { token },
      transports: ['websocket', 'polling']
    });

    newSocket.on('connect', () => {
      setConnected(true);
      console.log('WebSocket connected');
    });

    newSocket.on('disconnect', () => {
      setConnected(false);
      console.log('WebSocket disconnected');
    });

    setSocket(newSocket);

    return () => {
      newSocket.close();
    };
  }, [token]);

  return { socket, connected };
}

```

7.3 Join Chat Room

Before sending/receiving messages, you must join a chat room.

Event: `joinChat`

Emit:

```
socket.emit('joinChat', {
  chatId: 'chat-uuid-here'
});
```

Response Event: `chatJoined`

```
socket.on('chatJoined', (data) => {
  console.log('Joined chat:', data);
  // data = { chatId: 'chat-uuid', message: 'Joined chat successfully' }
});
```

Error Event: `error`

```
socket.on('error', (error) => {
  console.error('Socket error:', error);
  // error = { message: 'Error description' }
});
```

7.4 Send Message

Event: `sendMessage`

Emit:

```
socket.emit('sendMessage', {
  chatId: 'chat-uuid-here',
  content: 'Hello! I am interested in this property.'
});
```

Success Response: The message will be broadcast to all participants in the chat room via the `newMessage` event.

7.5 Receive Messages

Event: `newMessage`

Listen for incoming messages in real-time.

```
socket.on('newMessage', (message) => {
  console.log('New message received:', message);

  // Add message to chat UI
  addMessageToChat(message);
});
```

Message Object Structure:

```
{
  "id": "message-uuid",
  "content": "Hello! I am interested in this property.",
  "chatId": "chat-uuid",
  "senderId": "user-uuid",
  "sender": {
    "id": "user-uuid",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "createdAt": "2024-01-01T12:30:00.000Z"
}
```

7.6 Leave Chat Room

Event: `leaveChat`

Emit:

```
socket.emit('leaveChat', {
  chatId: 'chat-uuid-here'
});
```

Response Event: `chatLeft`

```
socket.on('chatLeft', (data) => {
  console.log('Left chat:', data);
  // data = { chatId: 'chat-uuid', message: 'Left chat successfully' }
});
```

7.7 Complete Chat Component Example (React)

```
import React, { useState, useEffect, useRef } from 'react';
import { io } from 'socket.io-client';

function ChatComponent({ chatId, token }) {
  const [socket, setSocket] = useState(null);
  const [messages, setMessages] = useState([]);
  const [newMessage, setNewMessage] = useState('');
  const [connected, setConnected] = useState(false);
  const messagesEndRef = useRef(null);

  // Initialize socket connection
  useEffect(() => {
    const newSocket = io('https://bricklyrealestate-production.up.railway.app', {
      auth: { token },
      transports: ['websocket', 'polling']
    });

    newSocket.on('connect', () => {
      setConnected(true);
      console.log('Connected to chat server');

      // Join chat room
      newSocket.emit('joinChat', { chatId });
    });

    newSocket.on('disconnect', () => {
      setConnected(false);
      console.log('Disconnected from chat server');
    });
  });
```

```

newSocket.on('chatJoined', (data) => {
  console.log('Joined chat:', data);
});

newSocket.on('newMessage', (message) => {
  setMessages(prev => [...prev, message]);
  scrollToBottom();
});

newSocket.on('error', (error) => {
  console.error('Socket error:', error);
  alert(error.message);
});

setSocket(newSocket);

return () => {
  if (newSocket) {
    newSocket.emit('leaveChat', { chatId });
    newSocket.close();
  }
};
}, [chatId, token]);

// Load initial messages from REST API
useEffect(() => {
  fetch(`https://bricklyrealestate-production.up.railway.app/chats/${chatId}/messages`, {
    headers: {
      'Authorization': `Bearer ${token}`
    }
  })
  .then(res => res.json())
  .then(data => setMessages(data))
  .catch(err => console.error('Error loading messages:', err));
}, [chatId, token]);

const scrollToBottom = () => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
};

const sendMessage = (e) => {
  e.preventDefault();

  if (!newMessage.trim() || !socket || !connected) return;

  socket.emit('sendMessage', {
    chatId,
    content: newMessage.trim()
  });

  setNewMessage('');
};

return (
  <div className="chat-container">
    <div className="chat-header">
      <h3>Chat</h3>
      <span className={connected ? 'status-online' : 'status-offline'}>
        {connected ? '● Online' : '○ Offline'}
      </span>
    </div>

    <div className="messages-container">
      {messages.map((msg) => (
        <div

```



```

    <div
      key={msg.id}
      className={`message ${msg.senderId === currentUserId ? 'sent' : 'received'}`}
    >
      <div className="message-sender">{msg.sender.name}</div>
      <div className="message-content">{msg.content}</div>
      <div className="message-time">
        {new Date(msg.createdAt).toLocaleTimeString()}
      </div>
    </div>
  </div>
  <div ref={messagesEndRef} />
</div>

<form onSubmit={sendMessage} className="message-input-form">
  <input
    type="text"
    value={newMessage}
    onChange={(e) => setNewMessage(e.target.value)}
    placeholder="Type a message..."
    disabled={!connected}
  />
  <button type="submit" disabled={!connected || !newMessage.trim()}>
    Send
  </button>
</form>
</div>
);
}

export default ChatComponent;

```

7.8 WebSocket Events Summary

Event	Direction	Description
connect	Server → Client	Connection established
disconnect	Server → Client	Connection closed
joinChat	Client → Server	Join a chat room
chatJoined	Server → Client	Successfully joined chat
leaveChat	Client → Server	Leave a chat room
chatLeft	Server → Client	Successfully left chat
sendMessage	Client → Server	Send a new message
newMessage	Server → Client	Receive a new message
error	Server → Client	Error occurred

7.9 WebSocket Best Practices

1. **Authentication:** Always include JWT token in socket connection
2. **Reconnection:** Handle reconnection automatically (socket.io does this by default)
3. **Error Handling:** Listen for `error` events and handle gracefully

4. **Room Management:** Always join chat room before sending messages
 5. **Cleanup:** Disconnect socket when component unmounts
 6. **Loading State:** Show loading indicator while connecting
 7. **Offline Handling:** Disable send button when disconnected
 8. **Message Persistence:** Load initial messages from REST API, then use WebSocket for real-time updates
-

7.10 Testing WebSocket Connection

Using Browser Console:

```
// Connect
const socket = io('https://bricklyrealestate-production.up.railway.app', {
  auth: { token: 'your-jwt-token' }
});

// Listen for connection
socket.on('connect', () => console.log('Connected!'));

// Join chat
socket.emit('joinChat', { chatId: 'your-chat-id' });

// Listen for messages
socket.on('newMessage', (msg) => console.log('New message:', msg));

// Send message
socket.emit('sendMessage', {
  chatId: 'your-chat-id',
  content: 'Test message'
});
```

Error Handling

All endpoints return standard HTTP status codes:

- 200 : Success
- 201 : Created
- 400 : Bad Request (validation errors)
- 401 : Unauthorized (missing/invalid token)
- 403 : Forbidden (insufficient permissions)
- 404 : Not Found
- 500 : Internal Server Error

Error Response Format:

```
{
  "statusCode": 400,
  "message": "Validation failed",
  "error": "Bad Request"
}
```

9. Frontend Integration Examples

9.1 Authentication Flow

```

// Login
async function login(email, password) {
  try {
    const response = await fetch('https://bricklyrealestate-production.up.railway.app/auth/login', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ email, password })
    });

    if (!response.ok) {
      const error = await response.json();
      throw new Error(error.message);
    }

    const data = await response.json();

    // Store token and user data
    localStorage.setItem('access_token', data.access_token);
    localStorage.setItem('user', JSON.stringify(data.user));

    return data;
  } catch (error) {
    console.error('Login error:', error);
    throw error;
  }
}

// Get stored token
function getToken() {
  return localStorage.getItem('access_token');
}

// Logout
function logout() {
  localStorage.removeItem('access_token');
  localStorage.removeItem('user');
  // Redirect to login page
}

```

9.2 Fetching Properties with Filters

```

async function fetchProperties(filters = {}) {
  const params = new URLSearchParams();

  // Add filters to query params
  if (filters.page) params.append('page', filters.page);
  if (filters.limit) params.append('limit', filters.limit);
  if (filters.type) params.append('type', filters.type);
  if (filters.minPrice) params.append('minPrice', filters.minPrice);
  if (filters.maxPrice) params.append('maxPrice', filters.maxPrice);
  if (filters.bedrooms) params.append('bedrooms', filters.bedrooms);
  if (filters.city) params.append('city', filters.city);
  if (filters.status) params.append('status', filters.status);

  try {
    const response = await fetch(
      `https://bricklyrealestate-production.up.railway.app/properties?${params}`,
      {
        headers: {
          'Content-Type': 'application/json'
        }
      }
    );

    if (!response.ok) throw new Error('Failed to fetch properties');

    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Error fetching properties:', error);
    throw error;
  }
}

// Usage
const properties = await fetchProperties({
  page: 1,
  limit: 12,
  type: 'VILLA',
  minPrice: 1000000,
  maxPrice: 5000000,
  city: 'Dubai',
  status: 'APPROVED'
});

```

9.3 Creating a Property with Images + AI Price Prediction

```

async function createProperty(formFields, imageFiles) {
  const token = getToken();

  // Validate minimum 4 images
  if (!imageFiles || imageFiles.length < 4) {
    throw new Error('Please select at least 4 images');
  }

  const formData = new FormData();

  // Append images
  imageFiles.forEach(file => formData.append('images', file));

  // Append all property fields
  Object.entries(formFields).forEach(([key, value]) => {
    formData.append(key, value);
  });
}

```

```

    },
  },
  try {
    const response = await fetch(
      'https://bricklyrealestate-production.up.railway.app/properties',
      {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${token}`
          // ⚠ Do NOT set Content-Type – browser sets it automatically with boundary
        },
        body: formData
      }
    );

    if (!response.ok) {
      const error = await response.json();
      throw new Error(error.message);
    }

    const property = await response.json();

    // Show AI price comparison
    if (property.aiPriceSuggested) {
      const diff = property.aiPriceDifferencePercentage;
      if (Math.abs(diff) > 20) {
        alert(`Your price is ${diff.toFixed(1)}% different from market average`);
      }
    }

    return property;
  } catch (error) {
    console.error('Error creating property:', error);
    throw error;
  }
}

// Usage with React file input
function CreatePropertyForm() {
  const [images, setImages] = useState([]);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const formFields = {
      title: e.target.title.value,
      description: e.target.description.value,
      type: e.target.type.value,
      price: e.target.price.value,
      bedrooms: e.target.bedrooms.value,
      bathrooms: e.target.bathrooms.value,
      area: e.target.area.value,
      furnished: e.target.furnished.checked,
      level: e.target.level.value,
      compound: e.target.compound.value,
      paymentOption: e.target.paymentOption.value,
      deliveryDate: e.target.deliveryDate.value,
      city: e.target.city.value,
      neighborhood: e.target.neighborhood.value,
      address: e.target.address.value,
      latitude: e.target.latitude.value,
      longitude: e.target.longitude.value,
    };

    await createProperty(formFields, images);
  };
}

```

```

return (
  <form onSubmit={handleSubmit}>
    {/* ... other fields ... */}
    <input
      type="file"
      multiple
      accept="image/*"
      onChange={(e) => setImages(Array.from(e.target.files))}
    />
    <p>Selected: {images.length} images (minimum 4 required)</p>
    <button type="submit" disabled={images.length < 4}>
      Create Property
    </button>
  </form>
);
}

```

9.4 Axios Configuration (Recommended)

```

import axios from 'axios';

// Create axios instance
const api = axios.create({
  baseURL: 'https://bricklyrealestate-production.up.railway.app',
  headers: {
    'Content-Type': 'application/json'
  }
});

// Add token to requests
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('access_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Handle errors
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Token expired or invalid
      localStorage.removeItem('access_token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

// Usage examples
const properties = await api.get('/properties', {
  params: { page: 1, limit: 10, type: 'VILLA' }
});

const newProperty = await api.post('/properties', propertyData);

```

10. Important Notes

10.1 AI Price Prediction

- AI price suggestion is **optional** and may be `null` if the AI service is unavailable
- Don't block property creation if AI prediction fails
- Use AI suggestion as a **guide**, not a strict requirement
- Show warnings if user price differs significantly (>20%) from AI suggestion

10.2 Property Status Flow

1. **PENDING** - Initial status after creation (requires admin approval)
2. **APPROVED** - Admin approved, visible to all users
3. **REJECTED** - Admin rejected, not visible to public

10.3 WebSocket Connection

- Always authenticate with JWT token
- Join chat room before sending messages
- Handle reconnection automatically
- Load initial messages from REST API, then use WebSocket for real-time updates

10.4 Seller Information

- When fetching properties, seller object includes only: `id`, `email`, `name`, `phone`
- Full user details are not exposed for privacy

11. Quick Reference

Base URLs

- **REST API:** `https://bricklyrealestate-production.up.railway.app`
- **WebSocket:** `wss://bricklyrealestate-production.up.railway.app`

Property Types

- `APARTMENT`, `VILLA`, `TOWNHOUSE`, `SINGLE_FAMILY`, `CONDO`, `DUPLEX`

Property Status

- `PENDING`, `APPROVED`, `REJECTED`

Booking Status

- `PENDING`, `CONFIRMED`, `CANCELLED`, `COMPLETED`

User Roles

- `BUYER`, `SELLER`, `ADMIN`

Last Updated: April 2026

API Version: 1.0.0

Documentation: Complete with AI Integration & WebSocket Support